

## Prompt Given

\*Code in C

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

• Test File format:

Command Description

EMAIL <sender category>,  
<subject line>,  
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

## Chat Gpt response

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define MAX_LINE 500

// Represents one email
typedef struct {
    char sender[50];
    char subject[300];
    char date[11];
} Email;

// MaxHeap structure
typedef struct {
    Email emails[MAX_EMAILS];
    int size;
} MaxHeap;

// -----
// Return priority based on sender category
// Higher number = higher priority
// -----
int senderPriority(char sender[]) {

    if (strcmp(sender, "Boss") == 0) {
        return 5;
    }
    else if (strcmp(sender, "Subordinate") == 0) {
        return 4;
    }
    else if (strcmp(sender, "Peer") == 0) {
        return 3;
    }
    else if (strcmp(sender, "ImportantPerson") == 0) {
        return 2;
    }
}
```

```

    }
    else {
        return 1; // OtherPerson
    }
}

```

```

// -----
// Convert MM-DD-YYYY into a number YYYYMMDD
//
// Example:
// 01-03-2025 -> 20250103
// 12-20-2024 -> 20241220
//
// This makes newer dates have larger values.
// -----
int dateValue(char date[]) {

```

```

    int month;
    int day;
    int year;

```

```

    sscanf(date, "%d-%d-%d", &month, &day, &year);

```

```

    return year * 10000 + month * 100 + day;
}

```

```

// -----
// Determine if email a has higher priority than b
// -----
int higherPriority(Email a, Email b) {

```

```

    int priorityA = senderPriority(a.sender);
    int priorityB = senderPriority(b.sender);

```

```

// First compare sender categories
if (priorityA > priorityB) {
    return 1;
}

```

```

    if (priorityA < priorityB) {
        return 0;
    }

    // Same sender category, so compare dates
    return dateValue(a.date) > dateValue(b.date);
}

// -----
// Swap two emails
// -----
void swapEmails(Email *a, Email *b) {

    Email temp = *a;
    *a = *b;
    *b = temp;
}

// -----
// Move newly added email upward
// -----
void upHeap(MaxHeap *heap, int index) {

    while (index > 0) {

        int parent = (index - 1) / 2;

        if (higherPriority(heap->emails[index],
                        heap->emails[parent])) {

            swapEmails(&heap->emails[index],
                    &heap->emails[parent]);

            index = parent;
        }
        else {
            break;
        }
    }
}

```

```

    }
}
}

// -----
// Add email to MaxHeap
// -----
void insertEmail(MaxHeap *heap, Email email) {

    if (heap->size >= MAX_EMAILS) {
        return;
    }

    heap->emails[heap->size] = email;

    upHeap(heap, heap->size);

    heap->size++;
}

// -----
// Move root downward after removing an email
// -----
void downHeap(MaxHeap *heap, int index) {

    while (1) {

        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        // Check left child
        if (left < heap->size &&
            higherPriority(heap->emails[left],
                          heap->emails[largest])) {

            largest = left;
        }
    }
}

```

```

// Check right child
if (right < heap->size &&
    higherPriority(heap->emails[right],
                  heap->emails[largest])) {

    largest = right;
}

// If parent is already largest, stop
if (largest == index) {
    break;
}

swapEmails(&heap->emails[index],
           &heap->emails[largest]);

index = largest;
}
}

// -----
// READ
//
// Remove highest-priority email without displaying it
// -----
void readEmail(MaxHeap *heap) {

    // Nothing to read
    if (heap->size == 0) {
        return;
    }

    // Put last email at root
    heap->emails[0] = heap->emails[heap->size - 1];

    heap->size--;

    // Restore MaxHeap

```

```

    if (heap->size > 0) {
        downHeap(heap, 0);
    }
}

// -----
// NEXT
//
// Display highest-priority email without removing it
// -----
void nextEmail(MaxHeap *heap) {

    if (heap->size == 0) {
        return;
    }

    Email email = heap->emails[0];

    printf("Next email:\n");
    printf("Sender: %s\n", email.sender);
    printf("Subject: %s\n", email.subject);
    printf("Date: %s\n", email.date);
}

// -----
// COUNT
// -----
void countEmails(MaxHeap *heap) {

    printf("There are %d emails to read.\n", heap->size);
}

// -----
// Remove newline from end of string
// -----
void removeNewline(char string[]) {

```

```

    string[strcspn(string, "\r\n")] = '\0';
}

// -----
// MAIN
// -----
int main(int argc, char *argv[]) {

    // Make sure filename was provided
    if (argc < 2) {
        printf("Usage: %s <input file>\n", argv[0]);
        return 1;
    }

    FILE *file = fopen(argv[1], "r");

    if (file == NULL) {
        printf("Could not open file.\n");
        return 1;
    }

    MaxHeap inbox;
    inbox.size = 0;

    char line[MAX_LINE];

    // Read file one line at a time
    while (fgets(line, sizeof(line), file) != NULL) {

        removeNewline(line);

        // -----
        // EMAIL command
        // -----
        if (strncmp(line, "EMAIL ", 6) == 0) {

            Email email;

            // Skip "EMAIL "

```

```

char *information = line + 6;

// First comma separates sender
char *sender = strtok(information, ",");

// Second field is subject
char *subject = strtok(NULL, ",");

// Third field is date
char *date = strtok(NULL, ",");

if (sender != NULL &&
    subject != NULL &&
    date != NULL) {

    strcpy(email.sender, sender);
    strcpy(email.subject, subject);
    strcpy(email.date, date);

    insertEmail(&inbox, email);
}

// -----
// NEXT command
// -----
else if (strcmp(line, "NEXT") == 0) {

    nextEmail(&inbox);
}

// -----
// READ command
// -----
else if (strcmp(line, "READ") == 0) {

    readEmail(&inbox);
}

// -----

```

```
// COUNT command
// -----
else if (strcmp(line, "COUNT") == 0) {

    countEmails(&inbox);
}
}

fclose(file);

return 0;
}
```

## Cluade's response

```
/*
 * Email Priority Queue for a busy CEO, implemented with a custom
 * list-based (dynamic array) MaxHeap built entirely from scratch in
 * plain C (no library heap/priority_queue routines used).
 *
 * Priority order (highest read first):
 *   Boss > Subordinate > Peer > ImportantPerson > OtherPerson
 * Ties within the same category are broken by date: newest first.
 *
 * Commands (one per line, read from a file given as argv[1], or
 * from stdin if no file is given):
 *   EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>
 *   NEXT
 *   READ
 *   COUNT
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_LINE    512
#define MAX_CATEGORY 32
#define MAX_SUBJECT 256
#define MAX_DATE    16

/* -----
 * Email record
 * ----- */
typedef struct {
    char category[MAX_CATEGORY];
    char subject[MAX_SUBJECT];
    char date[MAX_DATE]; /* stored as MM-DD-YYYY for display */
    long dateValue;      /* numeric YYYYMMDD form for comparisons */
} Email;

/* -----
```

```

* MaxHeap - list-based (dynamic array) implementation
* ----- */
typedef struct {
    Email *data;
    int size;
    int capacity;
} MaxHeap;

/* Returns a numeric rank for a category name. Higher = higher priority. */
int categoryRank(const char *category) {
    if (strcmp(category, "Boss") == 0)    return 5;
    if (strcmp(category, "Subordinate") == 0)    return 4;
    if (strcmp(category, "Peer") == 0)        return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0)    return 1;
    return 0; /* unknown category - lowest priority */
}

/* Converts "MM-DD-YYYY" into an integer YYYYMMDD so a larger value
 * always represents a later (newer) date. */
long dateToValue(const char *date) {
    int mm = 0, dd = 0, yyyy = 0;
    sscanf(date, "%d-%d-%d", &mm, &dd, &yyyy);
    return (long)yyyy * 10000L + (long)mm * 100L + (long)dd;
}

/* Returns 1 if email a should be considered HIGHER priority than
 * email b (i.e. a belongs closer to the root / front of the queue). */
int higherPriority(const Email *a, const Email *b) {
    int rankA = categoryRank(a->category);
    int rankB = categoryRank(b->category);
    if (rankA != rankB) {
        return rankA > rankB;
    }
    /* Same category: newer date wins (read first) */
    return a->dateValue > b->dateValue;
}

void heapInit(MaxHeap *heap) {
    heap->capacity = 8;
}

```

```

    heap->size = 0;
    heap->data = (Email *)malloc(sizeof(Email) * heap->capacity);
}

void heapFree(MaxHeap *heap) {
    free(heap->data);
    heap->data = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

void heapGrowIfNeeded(MaxHeap *heap) {
    if (heap->size >= heap->capacity) {
        heap->capacity *= 2;
        heap->data = (Email *)realloc(heap->data, sizeof(Email) * heap->capacity);
    }
}

void heapSwap(MaxHeap *heap, int i, int j) {
    Email temp = heap->data[i];
    heap->data[i] = heap->data[j];
    heap->data[j] = temp;
}

int heapParent(int i) { return (i - 1) / 2; }
int heapLeft(int i) { return 2 * i + 1; }
int heapRight(int i) { return 2 * i + 2; }

/* Moves the element at index i up until the heap property holds */
void siftUp(MaxHeap *heap, int i) {
    while (i > 0 && higherPriority(&heap->data[i], &heap->data[heapParent(i)])) {
        heapSwap(heap, i, heapParent(i));
        i = heapParent(i);
    }
}

/* Moves the element at index i down until the heap property holds */
void siftDown(MaxHeap *heap, int i) {
    int size = heap->size;
    while (1) {

```

```

    int left = heapLeft(i);
    int right = heapRight(i);
    int best = i;

    if (left < size && higherPriority(&heap->data[left], &heap->data[best])) {
        best = left;
    }
    if (right < size && higherPriority(&heap->data[right], &heap->data[best])) {
        best = right;
    }
    if (best == i) {
        break;
    }
    heapSwap(heap, i, best);
    i = best;
}
}

```

```

void heapInsert(MaxHeap *heap, Email e) {
    heapGrowIfNeeded(heap);
    heap->data[heap->size] = e;
    siftUp(heap, heap->size);
    heap->size++;
}

```

```

/* Returns a pointer to the highest priority email without removing
 * it, or NULL if the heap is empty. */
Email *heapPeekMax(MaxHeap *heap) {
    if (heap->size == 0) return NULL;
    return &heap->data[0];
}

```

```

/* Removes the highest priority email from the heap. Does nothing if
 * the heap is empty. */
void heapExtractMax(MaxHeap *heap) {
    if (heap->size == 0) return;
    heap->size--;
    heap->data[0] = heap->data[heap->size];
    if (heap->size > 0) {
        siftDown(heap, 0);
    }
}

```

```

    }
}

/* -----
 * String helpers
 * ----- */

/* Strips trailing '\n' / '\r' characters left by fgets */
void stripNewline(char *s) {
    int len = (int)strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r')) {
        s[--len] = '\0';
    }
}

/* Trims leading/trailing spaces in-place */
void trim(char *s) {
    int start = 0;
    int end = (int)strlen(s) - 1;
    while (isspace((unsigned char)s[start])) start++;
    while (end >= start && isspace((unsigned char)s[end])) end--;
    int len = end - start + 1;
    if (len > 0) {
        memmove(s, s + start, len);
    }
    s[len > 0 ? len : 0] = '\0';
}

/* -----
 * Command processing
 * ----- */

void processLine(char *line, MaxHeap *heap) {
    stripNewline(line);
    trim(line);
    if (strlen(line) == 0) return;

    if (strncmp(line, "EMAIL", 5) == 0) {
        /* Everything after "EMAIL " is "<category>,<subject>,<date>" */
        char *rest = line + 5;
        while (*rest == ' ') rest++;
    }
}

```

```

char *firstComma = strchr(rest, ',');
char *lastComma = strchr(rest, ',');
if (firstComma == NULL || lastComma == NULL || firstComma == lastComma) {
    return; /* malformed line - ignore */
}

Email e;
memset(&e, 0, sizeof(Email));

/* category: from rest up to firstComma */
int catLen = (int)(firstComma - rest);
if (catLen >= MAX_CATEGORY) catLen = MAX_CATEGORY - 1;
strncpy(e.category, rest, catLen);
e.category[catLen] = '\0';
trim(e.category);

/* subject: between firstComma+1 and lastComma */
int subLen = (int)(lastComma - (firstComma + 1));
if (subLen < 0) subLen = 0;
if (subLen >= MAX_SUBJECT) subLen = MAX_SUBJECT - 1;
strncpy(e.subject, firstComma + 1, subLen);
e.subject[subLen] = '\0';
trim(e.subject);

/* date: after lastComma */
strncpy(e.date, lastComma + 1, MAX_DATE - 1);
e.date[MAX_DATE - 1] = '\0';
trim(e.date);

e.dateValue = dateToValue(e.date);

heapInsert(heap, e);
}
else if (strcmp(line, "NEXT") == 0) {
    Email *top = heapPeekMax(heap);
    if (top == NULL) {
        printf("No emails to read.\n");
    } else {
        printf("Next email:\n");
    }
}

```

```

        printf("Sender: %s\n", top->category);
        printf("Subject: %s\n", top->subject);
        printf("Date: %s\n", top->date);
    }
}
else if (strcmp(line, "READ") == 0) {
    /* Removes the highest priority email without displaying it.
    * If the heap is empty, nothing happens. */
    heapExtractMax(heap);
}
else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap->size);
}
/* Unknown commands are silently ignored */
}

/* -----
* Main driver
* ----- */
int main(int argc, char *argv[]) {
    MaxHeap heap;
    heapInit(&heap);

    char line[MAX_LINE];
    FILE *input = stdin;

    if (argc > 1) {
        input = fopen(argv[1], "r");
        if (input == NULL) {
            fprintf(stderr, "Could not open file: %s\n", argv[1]);
            heapFree(&heap);
            return 1;
        }
    }

    while (fgets(line, sizeof(line), input) != NULL) {
        processLine(line, &heap);
    }

    if (input != stdin) {

```

```
        fclose(input);  
    }  
  
    heapFree(&heap);  
    return 0;  
}
```

# Analysis

## Correctness

To test the correctness of both programs, I threw different sample files at them to test their edge cases. These tests included attempting NEXT and Read on an empty heap, executing multiple NEXT commands without a READ, executing multiple READ commands in a row, comparing emails from different sender categories, comparing emails from the same category with different dates, testing dates across different years, using subject lines with spaces, putting emails in different priority orders, and giving both programs a file containing 1,000 emails. I could not get both programs to crash or give me a weird/incorrect output.

## Execution Time

Both programs are nearly identical in execution time. They both use the same  $O(\log n)$  binary-heap insert/remove.

## Space Complexity

The memory allocated for Maxheap in chatgpt code is allocated the moment MaxHeap inbox is declared in main. So because it is a local variable it lives on the stack not the heap. Chatgpt uses a fixed array size which is  $O(1)$ . Claude uses a dynamic array which is  $O(n)$ . But chatgpt has a hard cap of 1000. So for a CEO who never has more than 1000 emails at a time Chatgpt wins here but if the CEO was for some reason getting 10K emails, then Claude would have the edge.

## Maintainability

Chatgpt overall has a simpler approach to the problem, I had an easy time following its code and its comments. I feel like Claude overthought the solution a little and its code was difficult for me to follow. I believe Chatgpt provided more detailed comments as well. Since chatgpt made a simpler but effective program, I decided to choose its code. The only thing I decided to change was the hard email cap of 1000 to 10000.